Does Depth Hurt Plain Networks and Help Residual Ones? Reproducing the Degradation Claim of Deep Residual Learning on CIFAR-10

Royal Goronga
School of Computer Science and Applied Mathematics
University of the Witwatersrand, Johannesburg, South Africa
Code and all run logs: https://github.com/Royal93/resnet-repro

***Abstract*—He *et al.* [1] claim that deeper plain networks suffer a *degradation problem*, higher training and test error, while residual networks with parameter-free identity shortcuts improve with depth. I reimplemented their CIFAR-10 experiments (Section 4.2, Fig. 6, Table 6) in PyTorch and trained 23 models on a single laptop GPU. The qualitative claim reproduces clearly: plain-network test error rises from 9.56 ± 0.20% at 20 layers to 12.79 ± 0.41% at 56 layers (three seeds each), while ResNet error falls from 8.36 ± 0.27% at 20 layers to 6.50% at 110 layers, within the paper's reported spread. The quantitative match is weaker in the middle: with the paper's zero-padding (option A) shortcuts, my ResNet-56 is 0.96 points behind the paper's 6.97% and is beaten by ResNet-44, consistently over three seeds. Full-precision training does not change this. Replacing only the three dimension-changing shortcuts with projections (option B) brings ResNet-56 to 7.06 ± 0.33%, matching the paper, which suggests the gap lies in an unstated detail of the zero-padding shortcut.**

***Index Terms*—reproducibility, residual networks, degradation problem, CIFAR-10**

I. The Claim

The claim reproduced is the following, from Section 4.2 of [1]:

> *On CIFAR-10, plain networks of 20, 32, 44 and 56 layers show higher training and test error as depth increases, whereas residual networks with identity (option A) shortcuts, and therefore exactly the same depth, width and parameter count, show lower error as depth increases, down to 6.43% at 110 layers.*

The evidence in the paper is Fig. 6 (training and test curves for plain networks on the left and ResNets in the middle) and Table 6 (ResNet test errors of 8.75, 7.51, 7.17, 6.97 and 6.43% for 20, 32, 44, 56 and 110 layers). The authors support it with a controlled comparison: the networks are built from the same $6n$+2 template with 16, 32 and 64 filters, and the residual versions differ *only* by shortcuts that add no parameters. The plain networks' errors are shown only in Fig. 6, and only ResNet-110 is reported over several runs (best of five, mean $6.61 \pm 0.16\%$). Fig. 7 adds supporting analysis: residual layers have smaller response magnitudes than plain ones.

This claim was chosen because it is the paper's central argument, it can be tested at full scale on a 4 GB laptop GPU, and it is falsifiable in both directions: degradation could fail to appear in plain networks, or shortcuts could fail to remove it.

II. Route

I reimplemented the method from the paper's text in PyTorch [2]. The authors' Caffe [3] implementation was not used. The code (https://github.com/Royal93/resnet-repro) consists of a model definition (`models.py`), a training script following the paper's recipe (`train.py`), analysis scripts producing every table and figure in this report, and a single command, `reproduce.sh`, that runs everything.

The code was written with the help of an AI assistant. I chose the claim and the additional experiments in Section V, ran every training run on my own laptop, checked the architecture, schedule and augmentation against the paper, dealt with problems during training, and interpreted the results.

III. Setup

A. What matches the paper

The architecture is the paper's: a 3×3 convolution, then $3n$ residual blocks of two 3×3 convolutions over feature maps of 32, 16 and 8 pixels with 16, 32 and 64 filters, global average pooling and a 10-way classifier, for $6n$+2 weighted layers. Batch normalisation [4] follows each convolution, ReLU follows the shortcut addition, and weights use He initialisation [5]. Shortcuts use option A: identity, with zero-padded extra channels and stride 2 where the size halves. My parameter counts (0.27M, 0.46M, 0.66M, 0.85M, 1.73M) match Table 6, and plain and residual networks have identical counts.

Training follows the paper exactly: SGD with momentum 0.9, weight decay 0.0001, batch size 128, learning rate 0.1 divided by 10 at 32k and 48k iterations, and stopping at 64k iterations. Augmentation is the 4-pixel padding, random 32×32 crop and horizontal flip of [6], with per-pixel mean subtraction. Models train on all 50k training images and are tested on a single view of the 10k test images. ResNet-110 uses the paper's warm-up: learning rate 0.01 until training error falls below 80%.

TABLE I
DIFFERENCES FROM THE ORIGINAL SETUP

Aspect	Paper	This reproduction
Framework	Caffe	PyTorch
Hardware	2 GPUs	1 RTX 3050 laptop GPU (4 GB)
Precision	FP32	FP16 mixed precision [7]; one FP32 control run
Determinism	not stated	non-deterministic cuDNN kernels
Reported error	not stated (best of 5 for ResNet-110)	final error after 64k iterations
Runs per model	one value reported (5 runs for ResNet-110)	3 seeds for depths 20, 44*, 56; 1 for 32, 110
Plain-net numbers	Fig. 6 only	tabulated

*Plain-44 has one seed.

TABLE II
CIFAR-10 TEST ERROR (%) BY DEPTH. MEAN ± STD OVER 3 SEEDS; †SINGLE RUN.

Depth	Params	Plain (ours)	ResNet (ours)	ResNet (paper)
20	0.27M	9.56 ± 0.20	8.36 ± 0.27	8.75
32	0.46M	9.89†	7.78†	7.51
44	0.66M	10.94†	7.41 ± 0.30	7.17
56	0.85M	12.79 ± 0.41	7.93 ± 0.23	6.97
110	1.73M	–	6.50†	6.43

B. What I changed

Table I lists every known difference. The scale is *not* reduced: the dataset, model sizes and schedule are the paper's. Only the hardware, framework and numerical precision differ.

IV. Results Next to the Paper's

Table II compares final test errors like for like: same dataset, depths, parameter counts and schedule. Fig. 1 plots the same numbers against depth, and Fig. 2 shows the training curves in the style of the paper's Fig. 6.

Plain networks degrade. Test error rises with every increase in depth: 9.56 ± 0.20, 9.89, 10.94 and 12.79 ± 0.41%. The difference between 20 and 56 layers is far larger than the seed-to-seed spread. As in the paper, the deeper plain networks also have higher *training* error throughout (Fig. 2, left; final training error 1.21% at 20 layers against 5.53% at 56 layers), so the effect is not overfitting. The test-error gap between plain-56 and plain-20 is 3.23 points (Welch's $t = 12.2$).

Residual networks benefit from depth. ResNet test error falls from 8.36 ± 0.27% at 20 layers to 6.50% at 110 layers. ResNet-110 is within the paper's own spread (6.61 ± 0.16%), and ResNet-20 is 0.39 points better than the paper's value. At every depth the residual network beats the plain network with the same parameters.

Layer responses. Fig. 3 reproduces Fig. 7 of the paper. The mean standard deviation of 3×3 layer responses is lower for ResNets than for plain networks of the same depth (0.455 against 0.608 at 56 layers) and falls with ResNet depth (0.752 at 20 layers to 0.336 at 110), matching the paper's observation that deeper residual layers modify the signal less. One detail does not match: my plain networks' mean response also falls with depth, which the paper's figure does not show clearly.

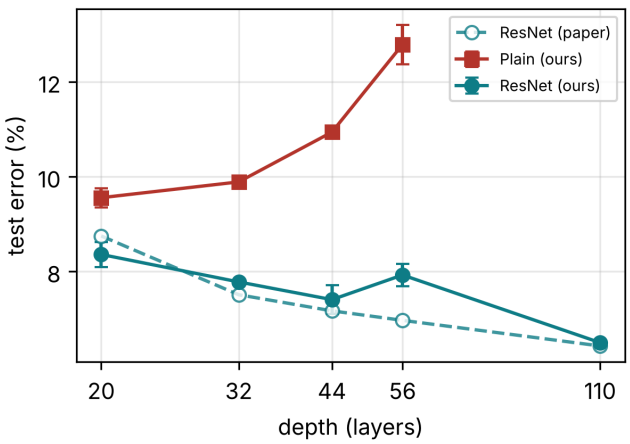


Fig. 1. Final test error against depth. Error bars are one standard deviation over three seeds. The paper's ResNet values (Table 6) are shown as open circles.

V. Numbers the Paper Does Not Report

Seed variability. The paper reports one run per model except ResNet-110. Over three seeds, plain-20 gives 9.56 ± 0.20% and ResNet-56 7.93 ± 0.23% (Tables II and III): seed-to-seed standard deviations of a few tenths of a point. This sets the scale against which any single-run difference in Table 6, such as the 0.20-point gap between ResNet-44 and ResNet-56, should be judged.

Tabulated plain-network errors for 20 to 56 layers (Table II); the paper shows these only as curves.

Projection shortcuts on CIFAR-10. The paper compares option A and option B shortcuts only on ImageNet (its Table 3). ResNet-56 with option B scores 7.06 ± 0.33% over three seeds, against 7.93 ± 0.23% with option A: 0.86 points better (Welch's $t = 3.7$), and within one standard deviation of the paper's option A value of 6.97%. Option B adds only about 0.01M parameters (three 1×1 convolutions), so the improvement is not explained by capacity. On ImageNet the paper found B only 0.5 points better than A for ResNet-34; on CIFAR-10 at 56 layers the difference in my runs is larger.

Precision. ResNet-56 trained in full FP32 scores 7.95% (one run), indistinguishable from the FP16 runs (7.93 ± 0.23%). Mixed precision therefore does not explain the gap; a single FP32 run cannot rule out a small effect, but any effect is smaller than the seed spread.

VI. The Gap

A. What did not reproduce

Two results do not match the paper.

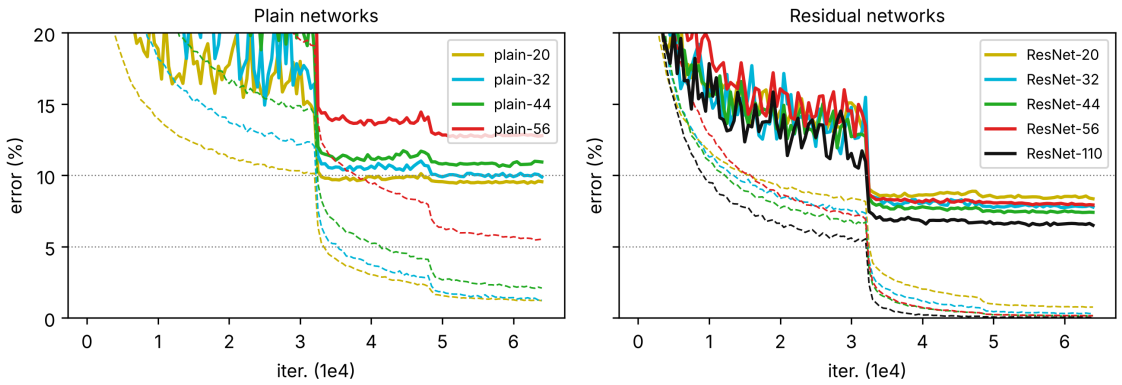


Fig. 2. Training on CIFAR-10, reproducing Fig. 6 of [1]. Dashed: training error (minibatches with augmentation). Bold: test error. Curves are averaged over the available seeds. Left: plain networks. Right: ResNets.

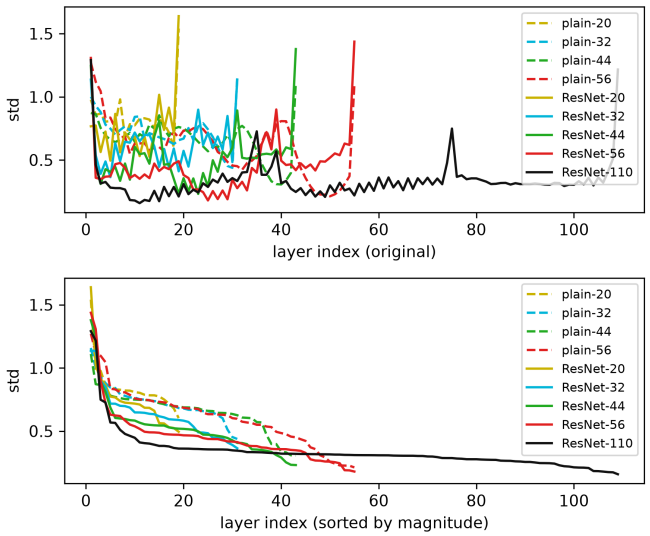


Fig. 3. Standard deviation of each 3×3 layer's response (after BN, before ReLU or addition) on the test set, seed 0. Top: original order. Bottom: sorted. Reproduces Fig. 7 of [1].

TABLE III
RESNET-44 AND RESNET-56 RUNS USED TO INVESTIGATE THE GAP (TEST ERROR, %)

Model and setting	Seeds	Each seed	Mean ± std
ResNet-44, option A, FP16 (main)	3	7.53, 7.06, 7.63	7.41 ± 0.30
ResNet-56, option A, FP16 (main)	3	8.04, 8.08, 7.66	7.93 ± 0.23
ResNet-56, option B, FP16	3	6.83, 7.44, 6.92	7.06 ± 0.33
ResNet-56, option A, FP32	1	7.95	7.95†

ResNet-56 is 0.96 points worse than reported (7.93 ± 0.23% against 6.97%). Even my best of three seeds, 7.66%, is short of it. ResNet-32 and ResNet-44 are also behind, by 0.27 and 0.24 points, whereas ResNet-20 and ResNet-110 match or beat the paper. The shortfall is therefore concentrated in the middle depths.

ResNet-56 does not beat ResNet-44. The paper's errors fall at every step in depth. In my runs, ResNet-44 (7.41 ± 0.30%) is better than ResNet-56 (7.93 ± 0.23%) by 0.52 points over three seeds each (Welch's $t = 2.4$). The difference is larger than either model's seed spread, though with only three seeds per model it is suggestive rather than conclusive. The overall trend from 20 to 110 layers still holds, but the claim that *each* additional block of depth helps does not reproduce at this step.

Neither failure is an optimisation failure. ResNet-56 ends with near-zero training error (0.15%), so it fits the training data; the gap is in how well it generalises.

B. What I tried

1) *More seeds.* Three seeds each for ResNet-44 and ResNet-56 test whether the ordering is noise. The ordering holds in the mean (ResNet-44 better by 0.52 points, larger than either seed spread), and the gap to the paper persists in every ResNet-56 seed (best 7.66%).
2) *Full precision.* Mixed precision is the largest numerical change from the paper. One FP32 run (7.95%) matches the FP16 mean, so precision is ruled out as the main cause.
3) *Projection shortcuts.* Option B gives the channels that option A zero-pads a learned path at the three points where the shape changes. This closes the gap: 7.06 ± 0.33% over three seeds against the paper's 6.97%.
4) *Checking the implementation.* Parameter counts match Table 6 exactly; the learning-rate schedule, augmentation and test protocol were checked against the paper line by line; and ResNet-20 and ResNet-110 matching the paper argues against a gross error in the architecture or training loop.

C. What I still cannot explain

The option B result locates the gap at the dimension-changing shortcuts, but not its cause. My option A follows the paper's description: subsample the input by taking every second pixel, then append zero channels. The paper does not specify how the subsampling is done (taking every second pixel or average pooling) or how the zero channels are placed,

and either could matter at the three points where half the shortcut's channels carry no signal. I did not have time to test these variants. Other unstated details could also contribute:

- *Reporting rule.* Table 6 gives the *best* of five runs for ResNet-110. If other entries are also best-of-k, my means are not like for like; my best seeds narrow but do not close the gap.
- *Batch-normalisation details.* Caffe and PyTorch update the running statistics with different default averaging constants, and the paper does not say whether weight decay applies to batch-normalisation parameters. I apply it to all parameters.
- *Padding order.* I subtract the mean before zero-padding, so padded pixels equal the mean colour. Padding raw images first would pad with a different value. The paper does not specify the order.
- *Multi-GPU batch normalisation.* With two GPUs, the paper's batch statistics were probably computed over 64 images per GPU rather than 128.

I still cannot explain why the shortfall affects 32 to 56 layers but not 20 or 110 layers with the same option A shortcuts. ResNet-110 used the warm-up schedule and ResNet-20 is shallow enough to be insensitive, but these are guesses; ResNet-32 and ResNet-110 also have only one seed each, so their agreement or disagreement with the paper is less certain than the 3-seed results.

D. Incidents

One run (plain-56, seed 1) was interrupted by a power loss that corrupted its log file. It was deleted and retrained from the start, so every reported run completed all 64k iterations without interruption.

VII. Data Ethics

CIFAR-10 [8] contains 60,000 32×32 images in ten object classes and has no person class, but people may appear incidentally in images. Its images were drawn from the 80 Million Tiny Images collection [9], which was gathered automatically from web searches without the consent of the photographers or of anyone pictured. That parent collection was withdrawn by its creators in 2020 after an audit found offensive labels and non-consensual images [10]. CIFAR-10's images were selected and relabelled by hand and the dataset remains available, but its distribution page states no formal licence. I used it only for research, did not redistribute it (the code downloads it from the official source), and the results here do not depend on any individual image.

VIII. Conclusion

The central claim reproduces: on CIFAR-10, deeper plain networks degrade, and identity shortcuts with no extra parameters reverse this, with ResNet-110 matching the paper. The finer-grained claim that ResNet error falls at every depth does not reproduce with zero-padding shortcuts: my option A ResNet-56 is 0.96 points short of the paper and worse than ResNet-44 across three seeds. Changing only the three dimension-changing shortcuts to projections restores the paper's number, which points to an under-specified detail of option A rather than to the residual principle itself. Seed spreads of a few tenths of a point, which the paper does not report, also suggest that its single-run comparisons between neighbouring depths should be read with caution.

References

[1] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.

[2] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

[3] Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell, "Caffe: Convolutional architecture for fast feature embedding," in *Proc. ACM Int. Conf. Multimedia*, 2014, pp. 675–678.

[4] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Machine Learning (ICML)*, 2015, pp. 448–456.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2015, pp. 1026–1034.

[6] C.-Y. Lee, S. Xie, P. Gallagher, Z. Zhang, and Z. Tu, "Deeply-supervised nets," in *Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2015, pp. 562–570.

[7] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2018.

[8] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.

[9] A. Torralba, R. Fergus, and W. T. Freeman, "80 million tiny images: A large data set for nonparametric object and scene recognition," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 30, no. 11, pp. 1958–1970, 2008.

[10] V. U. Prabhu and A. Birhane, "Large image datasets: A pyrrhic win for computer vision?" in *Proc. IEEE/CVF Winter Conf. Applications of Computer Vision (WACV)*, 2021, pp. 1536–1546.